# 实验四：网络传输实验（拥塞控制）实验报告

241275014 关翔文

2026 年 4 月 9 日

## 1 实验目的

1. 实现 TCP NewReno 拥塞控制机制。
2. 在给定拓扑下验证拥塞控制的正确性。
3. 记录拥塞窗口值，画出拥塞窗口的变化曲线图。

## 2 代码设计与实现

这次实验是在实验三的基础上加上 TCP NewReno 拥塞控制。主要的改动是 ACK 处理流程和超时处理，需要根据不同类型的 ACK（新 ACK、重复 ACK）来更新 `cwnd` 和 `ssthresh`，发送数据时实际的发送窗口取 `cwnd` 和接收方通告窗口的较小值。另外在连接建立后起了一个线程来定时把 `cwnd` 等状态写到文件里。

### 2.1 发送窗口与拥塞窗口的接入

为了让拥塞控制起作用，发送窗口 `snd_wnd` 不能只关注对方传送的 `adv_wnd` ，还要考虑自己的 `cwnd`，取两者的较小值。代码里对应的刷新函数如下：

Listing 1: 发送窗口刷新函数

```
static inline void tcp_refresh_send_window(struct tcp_sock *tsk)
{
    tsk->snd_wnd = min(tsk->cwnd, (u32)tsk->adv_wnd);

    if (tsk->adv_wnd == 0)
        tcp_set_persist_timer(tsk);
    else
        tcp_unset_persist_timer(tsk);
}
```

这个函数实现了两个功能：一是把 snd_wnd 设成 cwnd 和 adv_wnd 的较小值，这样就兼顾了流量控制和拥塞控制；二是当对方通告窗口为零时启动 persist 定时器，防止连接一直卡住。

## 2.2 基于 ACK 的拥塞控制逻辑

拥塞控制的核心在于根据 ACK 的类型更新发送端的状态。对于新的 ACK，发送端需要根据当前阶段执行慢启动或拥塞避免；对于重复 ACK，则需要统计重复次数，在达到 3 次后触发快速重传与快速恢复。

### 2.2.1 收到新的 ACK

收到一个新的 ACK 表示又有数据被对方确认了。如果现在正处在快恢复状态，就要关注 ACK 号是否越过了之前记的恢复点；越过去了就退出恢复状态，并把 cwnd 设回 ssthresh 附近。不在恢复状态的话，就按照 cwnd 和 ssthresh 的大小关系决定走慢启动还是拥塞避免。具体代码如下：

Listing 2: 收到新的 ACK 时的窗口调整

```
static inline void tcp_cc_on_new_ack(struct tcp_sock *tsk, struct tcp_cb *cb)
{
    if (tsk->c_state == TCP_CC_RECOVERY) {
        if (greater_or_equal_32b(cb->ack, tsk->recovery_point)) {
            tsk->cwnd = max(tsk->ssthresh, (u32)TCP_MSS);
            tsk->dup_ack_cnt = 0;
            tsk->c_state = TCP_CC_OPEN;
            tcp_refresh_send_window(tsk);
            return;
        } else {
            tsk->cwnd = tsk->ssthresh + TCP_MSS;
            tcp_refresh_send_window(tsk);
            tcp_retrans_send_buffer(tsk);
            return;
        }
    }

    tsk->dup_ack_cnt = 0;
    tsk->c_state = TCP_CC_OPEN;

    if (tsk->cwnd < tsk->ssthresh)
        tsk->cwnd += TCP_MSS;
```

```
    else
        tsk->cwnd += max((u32)1,
            ((u32)TCP_MSS * (u32)TCP_MSS) / max(tsk->cwnd, (u32)1));

    tcp_refresh_send_window(tsk);
}
```

从这段代码可以看出，在慢启动阶段，cwnd 每个 ACK 增加一个 MSS，增长得很快；一旦 cwnd 到了 ssthresh 以上，就改成每次增加大约 (MSS * MSS) / cwnd 的量，增长速度明显变慢。

### 2.2.2 收到重复 ACK

如果连续收到重复 ACK，说明可能中间有个包丢了，但后面的包对方收到了。代码里用 dup_ack_cnt 数着，数到 3 个重复 ACK 就触发快速重传和快速恢复。对应的处理逻辑如下：

Listing 3: 收到重复 ACK 时的窗口调整

```
static inline void tcp_cc_on_dup_ack(struct tcp_sock *tsk)
{
    tsk->dup_ack_cnt++;

    if (tsk->c_state == TCP_CC_RECOVERY) {
        tsk->cwnd += TCP_MSS;
        tcp_refresh_send_window(tsk);
        return;
    }

    if (tsk->dup_ack_cnt < 3) {
        tsk->c_state = TCP_CC_DISORDER;
        tcp_refresh_send_window(tsk);
        return;
    }

    u32 flight = tcp_cc_flight_size(tsk);
    tsk->ssthresh = max(flight / 2, (u32)(2 * TCP_MSS));
    tsk->cwnd = tsk->ssthresh + 3 * TCP_MSS;
    tsk->recovery_point = tsk->snd_nxt;
    tsk->c_state = TCP_CC_RECOVERY;

    tcp_refresh_send_window(tsk);
```

```
    tcp_retrans_send_buffer(tsk);
}
```

这里，重复 ACK 不到 3 个时只是把状态标成 TCP_CC_DISORDER，不做大动作；一旦到 3 个，就认为丢包了，把 ssthresh 设成当前在途数据量的一半（但至少 2 MSS），cwnd 调整成 ssthresh + 3 MSS，然后进入恢复状态并立即重传丢失的报文。

## 2.3 超时事件下的窗口回退

除了重复 ACK 之外，超时重传也会触发窗口调整。超时一般意味着网络拥塞比较严重，所以策略更保守：把 ssthresh 调小，cwnd 直接打回 1 MSS，重头开始慢启动。超时后的处理代码大致是这样：

Listing 4: 超时后的拥塞窗口回退

```
u32 flight = tsk->snd_nxt - tsk->snd_una;
tsk->ssthresh = max(flight / 2, (u32)(2 * TCP_MSS));
tsk->cwnd = TCP_MSS;
tsk->dup_ack_cnt = 0;
tsk->c_state = TCP_CC_OPEN;
tsk->snd_wnd = min(tsk->cwnd, (u32)tsk->adv_wnd);
```

## 2.4 cwnd 记录线程的设计

画 cwnd 变化曲线需要记录数据，所以在连接进入 ESTABLISHED 后启动了一个线程，每过 500us 把当时的 cwnd、ssthresh 等信息写进 cwnd.txt。线程函数长这样：

Listing 5: 拥塞窗口记录线程

```
static void *tcp_cwnd_thread(void *arg)
{
    struct tcp_sock *tsk = (struct tcp_sock *)arg;
    FILE *fp = fopen("cwnd.txt", "w");
    if (!fp)
        return NULL;

    int time_us = 0;
    while (tsk->state == TCP_ESTABLISHED && time_us < 1000000) {
        usleep(500);
        time_us += 500;
        fprintf(fp, "%d %u %u %u %u %d\n",
            time_us, tsk->cwnd, tsk->ssthresh,
```

```
            tsk->adv_wnd, tsk->snd_wnd, tsk->c_state);
        fflush(fp);
    }

    fclose(fp);
    return NULL;
}
```

连接建立好，检查 cwnd_record_on 标志，如果没启动记录线程就创建一个，而且用 detach 方式，不用回收。记录线程每 500us 写一行，总共记录 1 秒（实验里文件传输一般很快）。由于快恢复过程很短，图上通常只能看到窗口突然掉下来，中间的细节抓不到，这和实验说明里说的现象是一致的。

Listing 6: 连接建立后启动记录线程

```
if (tsk->state == TCP_ESTABLISHED && !tsk->cwnd_record_on) {
    pthread_t cwnd_record;
    tsk->cwnd_record_on = 1;
    pthread_create(&cwnd_record, NULL, tcp_cwnd_thread, (void *)tsk);
    pthread_detach(cwnd_record);
}
```

# 3 实验结果与分析

## 3.1 文件传输正确性验证

在丢包拓扑下完成文件传输测试后，使用以下命令比较客户端输入文件与服务器输出文件：

```
diff client-input.dat server-output.dat
```

结果无输出，说明客户端发送的文件与服务器接收的文件一致，表明程序在丢包环境下仍能正确完成可靠传输。

## 3.2 cwnd 曲线展示

图 1 为实验记录得到的 TCP NewReno 拥塞窗口变化曲线。

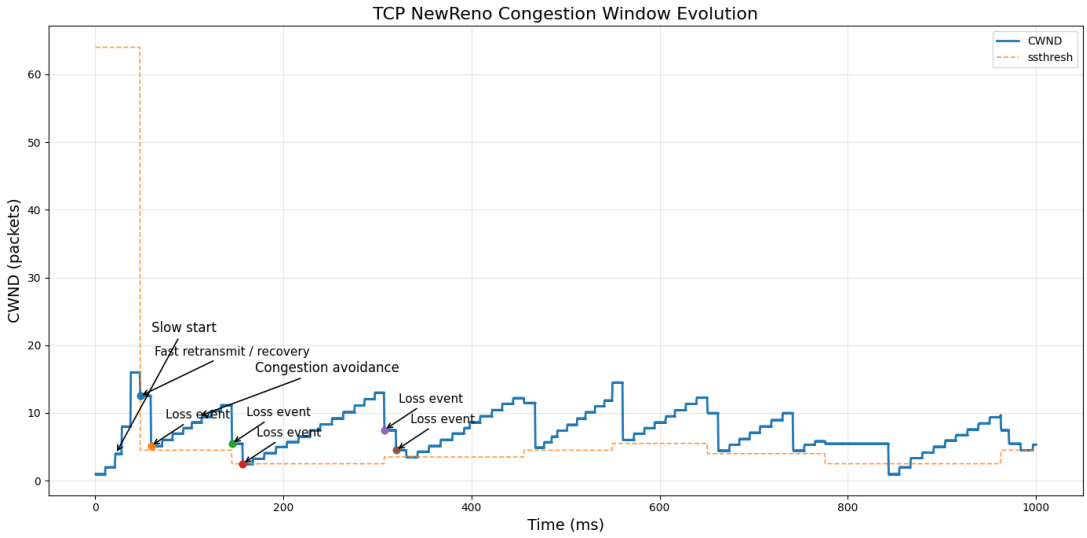


图 1: $TCPNewReno$ 拥塞窗口变化曲线

## 3.3 曲线分析

从图象信息中可以得出，一开始 cwnd 增长得很快，这是慢启动的效果；之后增速变平缓了，进入了拥塞避免。中间有几个地方 cwnd 突然减少，对应网络中发生了丢包。按 NewReno 的做法，这种减少一般是因为连续收到 3 个重复 ACK 触发了快重传和快恢复，或者发生了超时重传。因为每 500us 记录一次，快恢复的过程又很短，所以图上只能看到窗口骤降，看不到恢复阶段内部的调整细节，这跟实验要求里说的情况是一致的。

综上，本实验实现的 TCP NewReno 拥塞控制机制能够在丢包环境下正确完成文件传输，并在 `cwnd` 曲线中体现出慢启动、拥塞避免以及丢包后的窗口回退等典型特征，说明实验基本达到了预期目标。